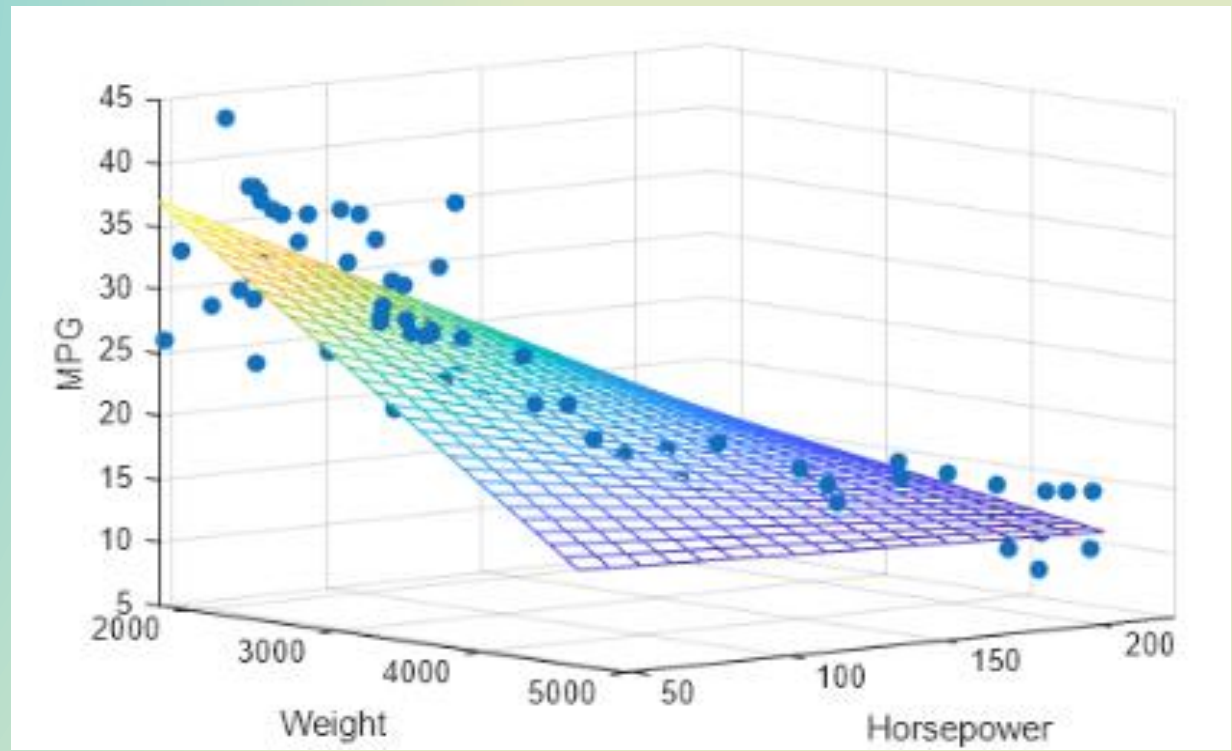
Regresie liniara

Gr. 11/1



Proiect – partea I :

Antonio Almasan

Culda Alisa

Motoc Amelie

Descrierea problemei

Problema abordata consta în aproximarea unei functii definite de doua variabile de intrare, x1 și x2, și o variabila de iesire y: f(x1, x2) = y

Se urmareste construirea unui model care sa aproximeze cat mai bine valorile masurate, folosind regresia liniara.

Modelul obtinut din datele de identificare este validat pe un set de date de validare, incercandu – se gasirea unei erori medii patratice cat mai mici, pentru un grad optim.

Structura aproximatorului și procedura de determinare a parametrilor

Aproximatorul folosește un polinom de ordin m de forma (1, x1, x2, x1^2, x1*x2, x2^2, ...), cu dimensiunea in functie de gradul m. Acesti termeni formeaza **matricea de regresori PHI,** unde fiecare linie corespunde unui set de date (x1, x2) din setul de identificare, matrice a carei termeni sunt cunoscuti, dar trebuie formati

Modelul aproximat va fi de forma **Y^ = PHI * THETA**, unde THETA este matricea coloana cu **parametrii** care trebuie obtinuti. Aceasta se obtine prin metoda celor mai mici patrate sau, in Matlab, cu ajutorul impartirii la stanga **THETA = PHI \ Y.**

Se urmareste obtinerea celei mai bune estimari a functiei prin gasirea gradului m optim.

Functia de generare a matricii de regresori:

```
function phi = generareRegresori(N1, N2,m, id)
    row = 1;
    for i = 1 : N1
        for j = 1 : N2
            k = 1; %contor pt randuri
            for p = 0 : m
                for q = 0 : (m - p)
                    phi(row, k) = id.X{1}(i)^p * id.X{2}(j)^q;
                    k = k + 1;
                end
            end
            row = row + 1;
        end
    end
end
```

Caracteristici esențiale ale soluției

- Solutia include o functie pentru calculul matricei de regresori, reglabila in functie de dimensiunile lui x1, x2, in functie de gradul m si de setul de date.

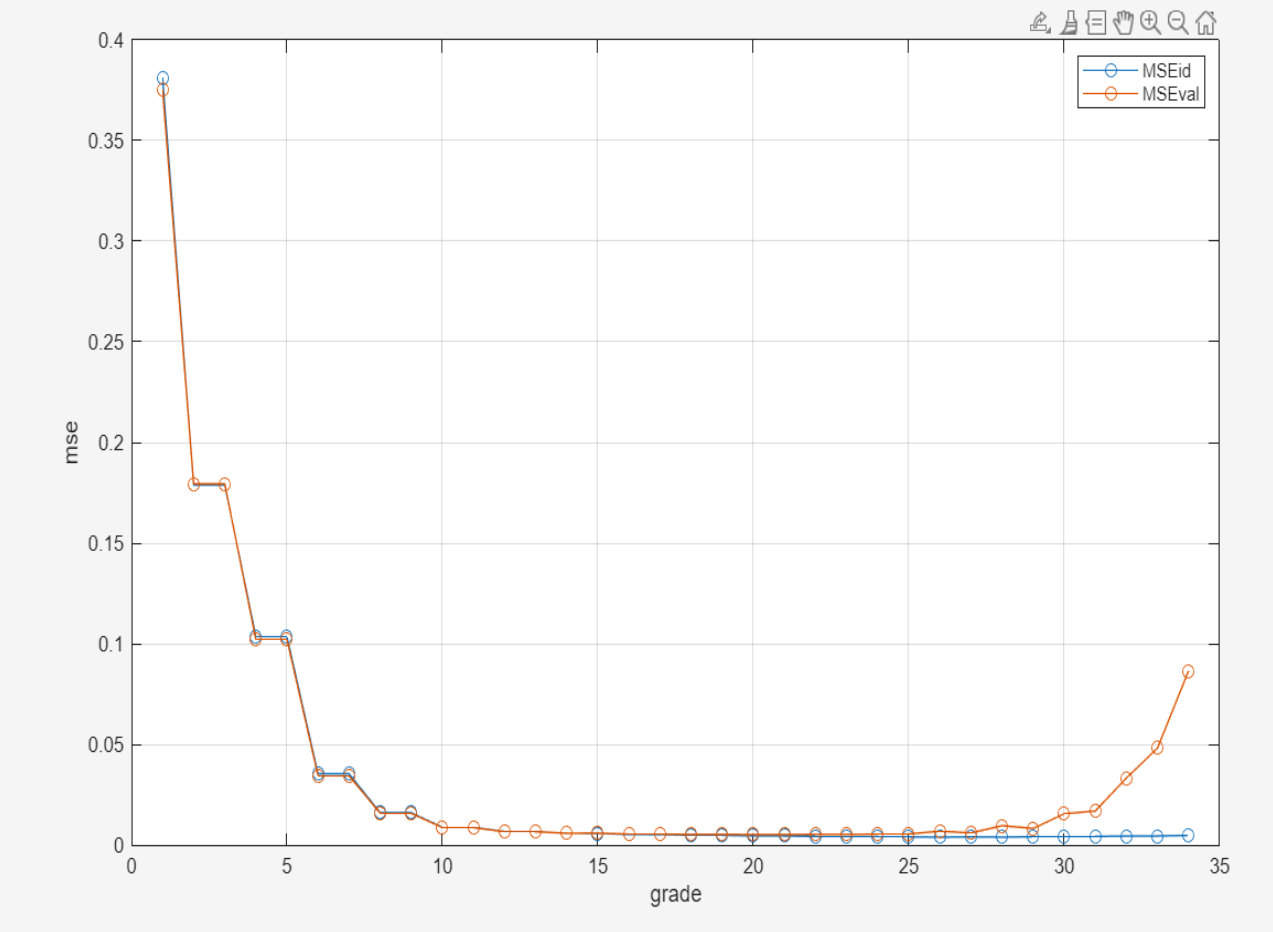
- In aceeasi bucla "for" care calculeaza erorile in functie de grad, solutia salveaza gradul optim **m_optim** pentru care MSE este minima pe datele de **validare** si salveaza si functia aproximata pentru acel m_optim. In acest sens, variabila mse_minim a primit o valoare foarte moare la initializare.

```
if mse_val(m) < mse_minim
    mse_minim = mse_val(m);
    grad_optim = m;
    Y_trans_optim = Y_trans;
    Y_transv_optim = Y_transv;
end
```

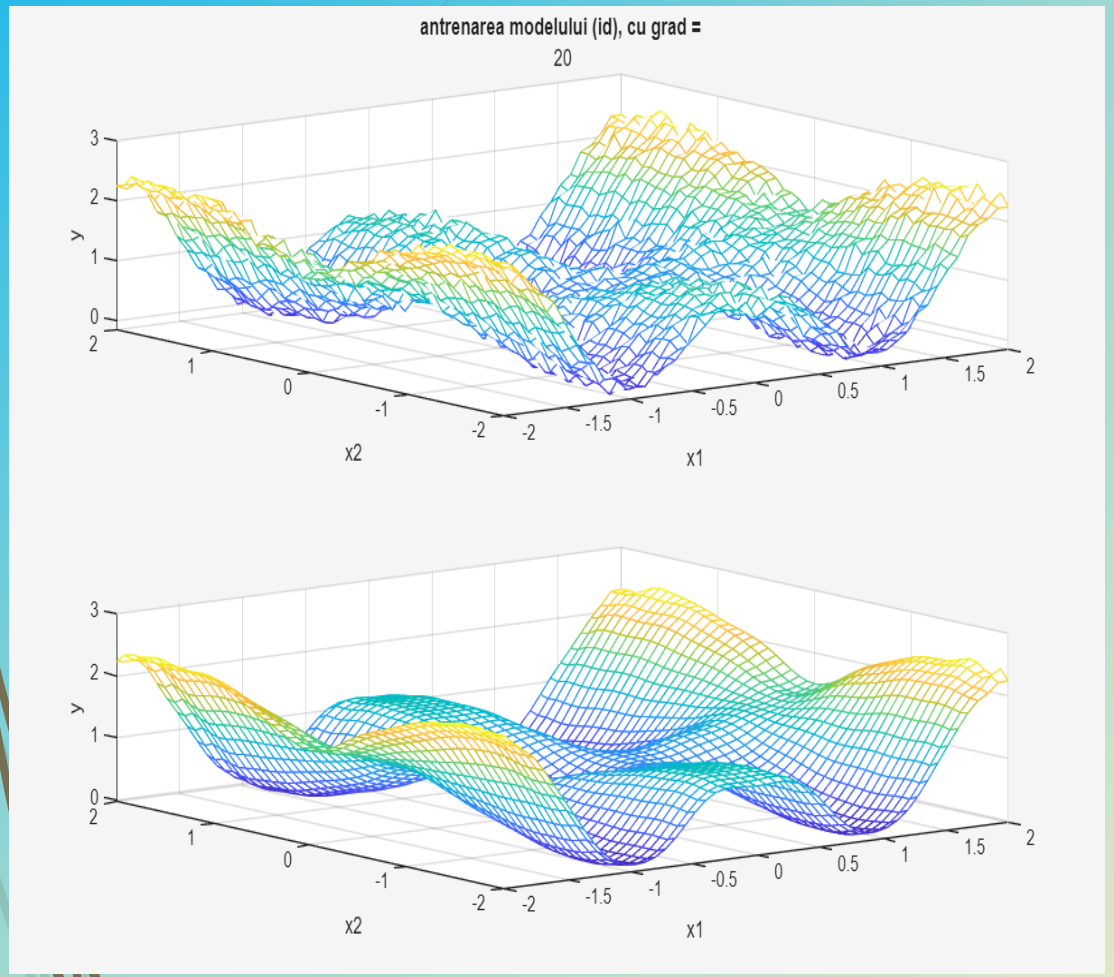
Rezultate de acordare (MSE în funcție de m)

Graficul confirma comportamentul asteptat:

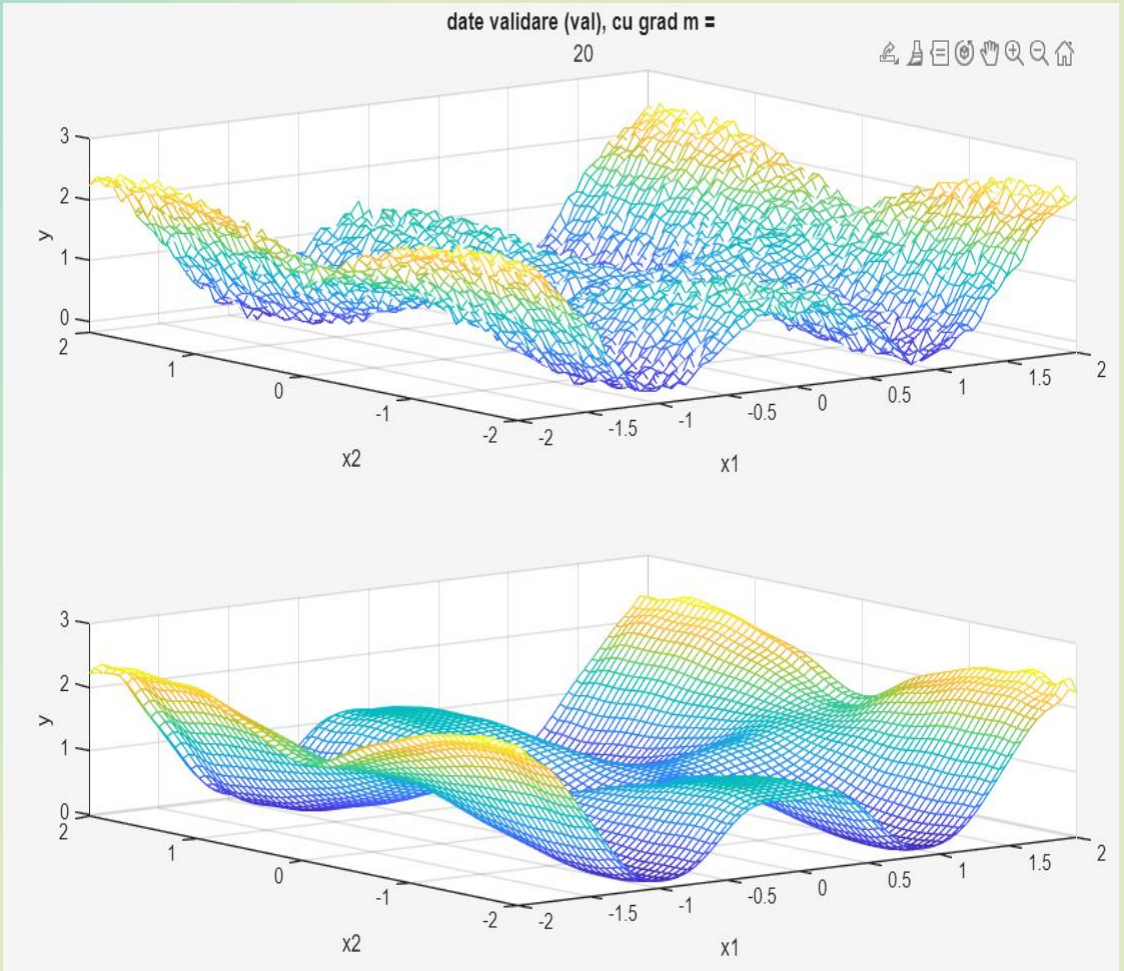
- Erorile sunt mari la început (fenomen de subantrenare);
- Pe masura ce gradul creste, ele ating un minim (modelul optim);
- Apoi cresc (fenomen de supraantrenare);



Graficele pentru m_optim = 20



Identificare



validare

Concluzii

Din graficul erorilor medii patratice, se observa ca MSE_id scade incontinuu, deoarece modelul invata si se potriveste tot mai bine pe datele de identificare. In schimb, MSE_val scade doar pana la un moment dat, iar apoi incepe sa creasca, deoarece modelul nu a invatat pe datele de validare si acestea sunt afectate de zgomot. Acest fenomen se numeste **supraantrenare** si se urmareste evitarea lui.

Asadar, MSE_val devine o metoda de a stabili performanta sistemului in functie de gradul ales. Este dorita cea mai mica valoare a MSE_val, la care va fi ales si gradul optim al modelului.

In acest fel, modelul va fi cea mai apropiata estimare a functiei reale, folosind regresia liniara.

Anexa – codul sursa:

```
clc
clear all
close all
load("proj_fit_11.mat");
N1 = length(id.X{1});
N2 = length(id.X{2});
v_N1 = length(val.X{1});
v_N2 = length(val.X{2});
grade = 1 : 34;
mse_minim = 10000000;
for m = grade
 phi_id = []; %se initializeaza matricea
 phi_id = generareRegresori(N1, N2, m, id);
 %construirea setului de antrenare
 Y = reshape(id.Y, [], 1);
 theta = phi_id \ Y;
 Y_trans = phi_id * theta;
 mse_id(m) = mean((Y_trans - Y).^2);
 %pentru validare
 phi_val = [];
 phi_val = generareRegresori(v_N1, v_N2, m, val);
 Yv = reshape(val.Y, [], 1);
 Y_transv = phi_val * theta;
 mse_val(m) = mean((Y_transv - Yv).^2);
 %salvam valori pentru gradul optim
 if mse_val(m) < mse_minim
  mse_minim = mse_val(m);
  grad_optim = m;
  Y_trans_optim = Y_trans;
  Y_transv_optim = Y_transv;
 end
end
```

```
figure
plot(grade, mse_id, '-o');
hold on;
plot(grade, mse_val, '-o');
xlabel('grade'); ylabel('mse'); legend('MSEid', 'MSEval');
grid on;
m=grad_optim;
figure
subplot(2, 1, 1)
mesh(id.X{1},id.X{2},id.Y);
title("antrenarea modelului (id), cu grad = ", m);
xlabel('x1'); ylabel('x2'); zlabel('y');
Y_id_optim = reshape(Y_trans_optim, N2, N1);
subplot(2, 1, 2)
mesh(id.X{1}, id.X{2}, Y_id_optim);
xlabel('x1'); ylabel('x2'); zlabel('y');
%pentru validare
figure
subplot(2, 1, 1)
mesh(val.X{1}, val.X{2}, val.Y);
title("date validare (val), cu grad m = ", m);
xlabel('x1'); ylabel('x2'); zlabel('y');
Y_val_optim = reshape(Y_transv_optim, v_N2, v_N1);
subplot(2, 1, 2)
mesh(val.X{1}, val.X{2}, Y_val_optim);
xlabel('x1'); ylabel('x2'); zlabel('y');
function phi = generareRegresori(N1, N2,m, id)
 row = 1;
 for i = 1 : N1
  for j = 1 : N2
   k = 1; %contor pt randuri
   for p = 0 : m
    for q = 0 : (m - p)
     phi(row, k) = id.X{1}(i)^p * id.X{2}(j)^q;
     k = k + 1;
    end
   end
   row = row + 1;
  end
 end
end
```